Supersetid :8003253

Digital Nurture 5.0 Deep Skilling Handbook Java FSE - Solution

ALGORITHMS_DATA STRUCTURES

Supersetid :8003253

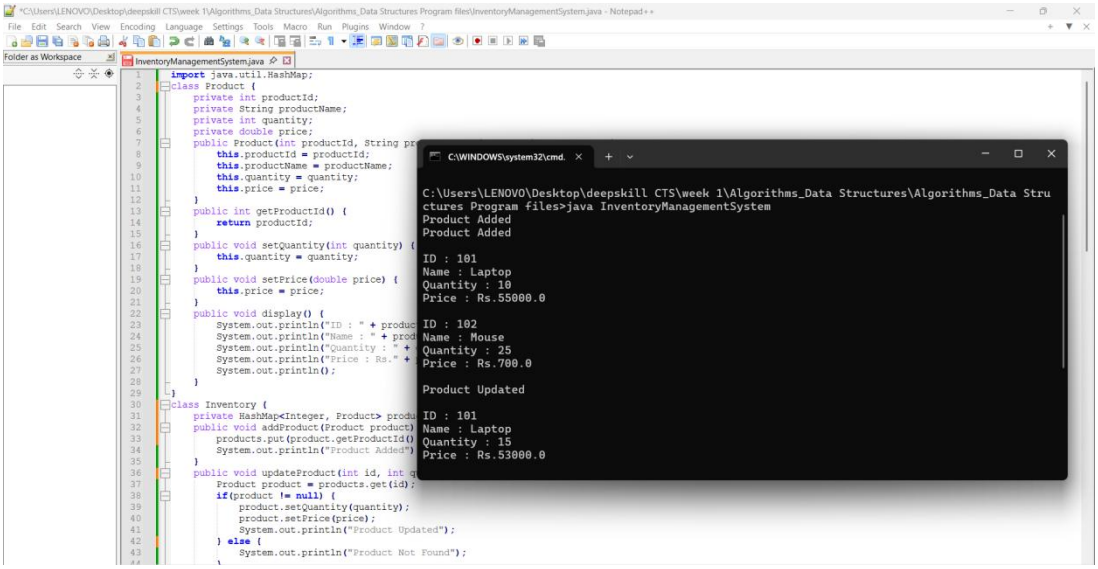
Exercise 1: Inventory Management System

```
import java.util.HashMap;
class Product {
   private int productId;
   private String productName;
   private int quantity;
   private double price;
        public Product(int productId, String productName, int quantity, double price) {
      this.productId = productId;
      this.productName = productName;
      this.quantity = quantity;
      this.price = price;
   }
        public int getProductId() {
      return productId;
   }
        public void setQuantity(int quantity) {
      this.quantity = quantity;
   }
        public void setPrice(double price) {
      this.price = price;
   }
        public void display() {
      System.out.println("ID : " + productId);
      System.out.println("Name : " + productName);
      System.out.println("Quantity : " + quantity);
      System.out.println("Price : Rs." + price);
      System.out.println();
   }
}
class Inventory {
   private HashMap<Integer, Product> products = new HashMap<>();
   public void addProduct(Product product) {
      products.put(product.getProductId(), product);
      System.out.println("Product Added");
   }
   public void updateProduct(int id, int quantity, double price) {
      Product product = products.get(id);
                if(product != null) {
                        product.setQuantity(quantity);
         product.setPrice(price);
                        System.out.println("Product Updated");
      } else {
         System.out.println("Product Not Found");
      }
   }
   public void deleteProduct(int id) {
                if(products.remove(id) != null) {
                        System.out.println("Product Deleted");
      } else {
                        System.out.println("Product Not Found");
      }
   }
   public void displayProducts() {
                for(Product product : products.values()) {
```

Supersetid :8003253

```
                product.display();
        }
    }
}
public class InventoryManagementSystem {

    public static void main(String[] args) {
        Inventory inventory = new Inventory();
        Product p1 = new Product(101,"Laptop",10,55000);
        Product p2 = new Product(102,"Mouse",25,700);
        inventory.addProduct(p1);
        inventory.addProduct(p2);
        System.out.println();
        inventory.displayProducts();
        inventory.updateProduct(101,15,53000);
        System.out.println();
        inventory.displayProducts();
        inventory.deleteProduct(102);
        System.out.println();
        inventory.displayProducts();

    }
}
```

Output:



Supersetid :8003253

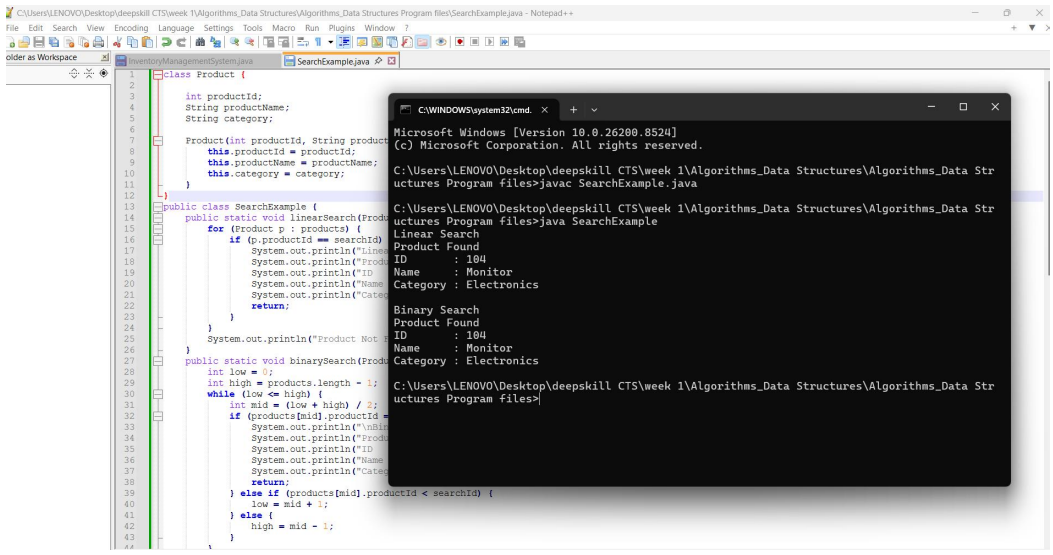
Exercise 2: E-commerce Platform Search Function

```
class Product {

    int productId;
    String productName;
    String category;

    Product(int productId, String productName, String category) {
        this.productId = productId;
        this.productName = productName;
        this.category = category;
    }
}
public class SearchExample {
    public static void linearSearch(Product[] products, int searchId) {
        for (Product p : products) {
            if (p.productId == searchId) {
                System.out.println("Linear Search");
                System.out.println("Product Found");
                System.out.println("ID       : " + p.productId);
                System.out.println("Name     : " + p.productName);
                System.out.println("Category : " + p.category);
                return;
            }
        }
        System.out.println("Product Not Found");
    }
    public static void binarySearch(Product[] products, int searchId) {
        int low = 0;
        int high = products.length - 1;
        while (low <= high) {
            int mid = (low + high) / 2;
            if (products[mid].productId == searchId) {
                System.out.println("\nBinary Search");
                System.out.println("Product Found");
                System.out.println("ID       : " + products[mid].productId);
                System.out.println("Name     : " + products[mid].productName);
                System.out.println("Category : " + products[mid].category);
                return;
            } else if (products[mid].productId < searchId) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }
        System.out.println("Product Not Found");
    }
    public static void main(String[] args) {
        Product[] products = {
                new Product(101, "Laptop", "Electronics"),
                new Product(102, "Mouse", "Accessories"),
                new Product(103, "Keyboard", "Accessories"),
                new Product(104, "Monitor", "Electronics"),
                new Product(105, "Printer", "Electronics")
        };
```

Supersetid :8003253

```
        Scanner sc = new Scanner(System.in);
        int searchId = sc.nextInt();
        linearSearch(products, searchId);
        binarySearch(products, searchId);
    }
}
```

OUTPUT:



Supersetid :8003253

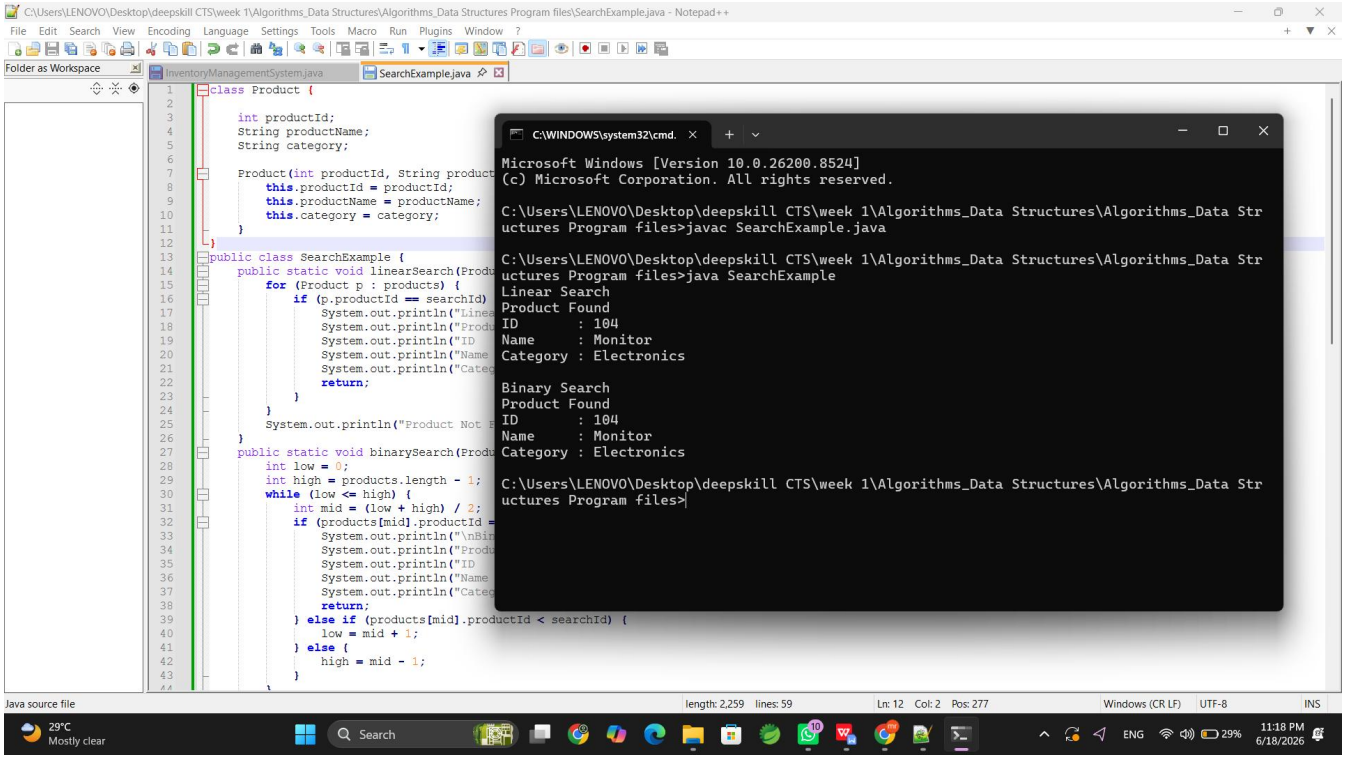
Exercise 3: Sorting Customer Orders

```
class Order {

    int orderId;
    String customerName;
    double totalPrice;
    Order(int orderId, String customerName, double totalPrice) {
        this.orderId = orderId;
        this.customerName = customerName;
        this.totalPrice = totalPrice;
    }
    public void display() {
        System.out.println(orderId + "  " + customerName + "  Rs." + totalPrice);
    }
}
public class SortingExample {
    public static void bubbleSort(Order[] orders) {
        int n = orders.length;
        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - i - 1; j++) {
                if (orders[j].totalPrice > orders[j + 1].totalPrice) {
                    Order temp = orders[j];
                    orders[j] = orders[j + 1];
                    orders[j + 1] = temp;
                }
            }
        }
    }
    public static void quickSort(Order[] orders, int low, int high) {
        if (low < high) {
            int pivot = partition(orders, low, high);
            quickSort(orders, low, pivot - 1);
            quickSort(orders, pivot + 1, high);
        }
    }
    public static int partition(Order[] orders, int low, int high) {
        double pivot = orders[high].totalPrice;
        int i = low - 1;
        for (int j = low; j < high; j++) {
            if (orders[j].totalPrice < pivot) {
                i++;
                Order temp = orders[i];
                orders[i] = orders[j];
                orders[j] = temp;
            }
        }
        Order temp = orders[i + 1];
        orders[i + 1] = orders[high];
        orders[high] = temp;
        return i + 1;
    }
    public static void displayOrders(Order[] orders) {
        for (Order order : orders) {
            order.display();
```

Supersetid :8003253

```
    }
  }
  public static void main(String[] args) {
    Order[] orders = {
        new Order(101, "Rahul", 2500),
        new Order(102, "Anjali", 8000),
        new Order(103, "Kiran", 1500),
        new Order(104, "Priya", 6000),
        new Order(105, "Amit", 4500)
    };
    bubbleSort(orders);
    System.out.println("Bubble Sort:");
    displayOrders(orders);
    Order[] orders2 = {
        new Order(101, "Rahul", 2500),
        new Order(102, "Anjali", 8000),
        new Order(103, "Kiran", 1500),
        new Order(104, "Priya", 6000),
        new Order(105, "Amit", 4500)
    };

    quickSort(orders2, 0, orders2.length - 1);

    System.out.println("\nQuick Sort:");
    displayOrders(orders2);
  }
}
```

OUTPUT:



Supersetid :8003253

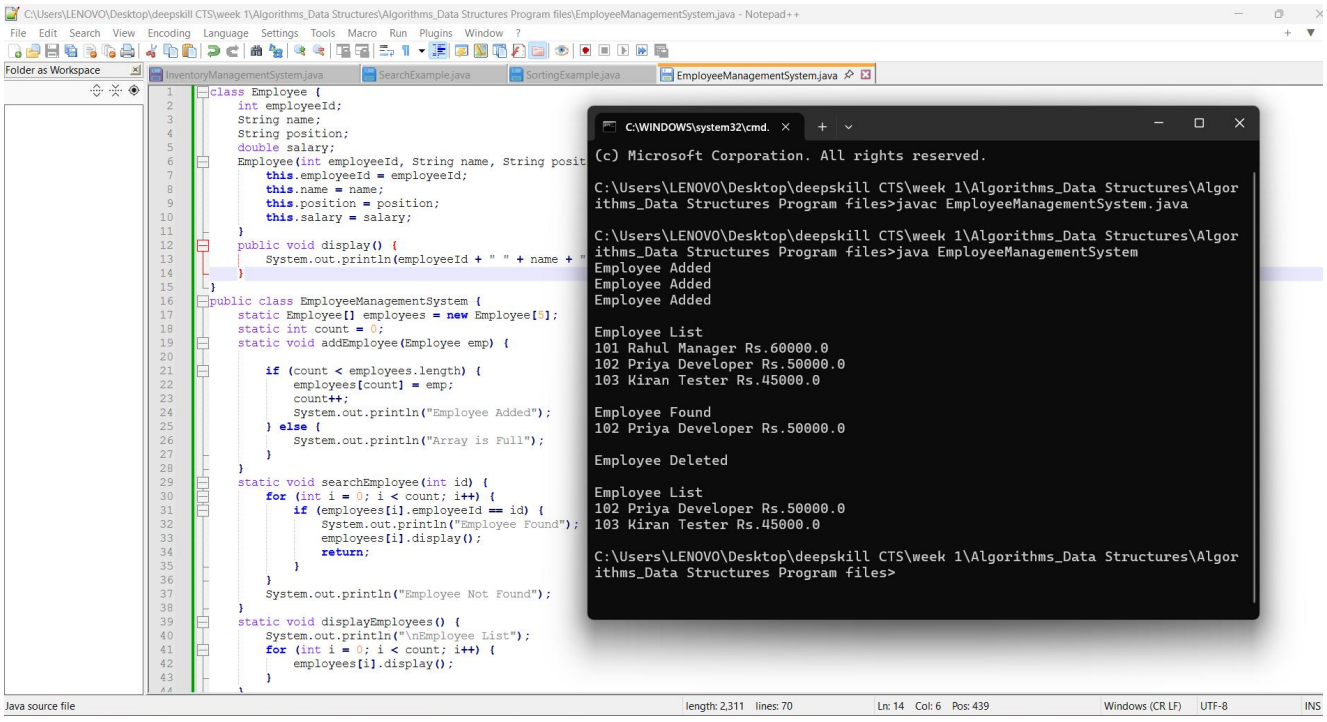
Exercise 4: Employee Management System

```
class Employee {
    int employeeId;
    String name;
    String position;
    double salary;
    Employee(int employeeId, String name, String position, double salary) {
        this.employeeId = employeeId;
        this.name = name;
        this.position = position;
        this.salary = salary;
    }
    public void display() {
        System.out.println(employeeId + " " + name + " " + position + " Rs." + salary);
    }
}
public class EmployeeManagementSystem {
    static Employee[] employees = new Employee[5];
    static int count = 0;
    static void addEmployee(Employee emp) {

        if (count < employees.length) {
            employees[count] = emp;
            count++;
            System.out.println("Employee Added");
        } else {
            System.out.println("Array is Full");
        }
    }
    static void searchEmployee(int id) {
        for (int i = 0; i < count; i++) {
            if (employees[i].employeeId == id) {
                System.out.println("Employee Found");
                employees[i].display();
                return;
            }
        }
        System.out.println("Employee Not Found");
    }
    static void displayEmployees() {
        System.out.println("\nEmployee List");
        for (int i = 0; i < count; i++) {
            employees[i].display();
        }
    }
    static void deleteEmployee(int id) {
        for (int i = 0; i < count; i++) {
            if (employees[i].employeeId == id) {
                for (int j = i; j < count - 1; j++) {
                    employees[j] = employees[j + 1];
                }
                employees[count - 1] = null;
                count--;
                System.out.println("Employee Deleted");
                return;
```

Supersetid :8003253

```
        }
    }
    System.out.println("Employee Not Found");
  }
  public static void main(String[] args) {
    addEmployee(new Employee(101, "Rahul", "Manager", 60000));
    addEmployee(new Employee(102, "Priya", "Developer", 50000));
    addEmployee(new Employee(103, "Kiran", "Tester", 45000));
    displayEmployees();
    System.out.println();
    searchEmployee(102);
    System.out.println();
    deleteEmployee(101);
    displayEmployees();
  }
}
```

OUTPUT:



Supersetid :8003253

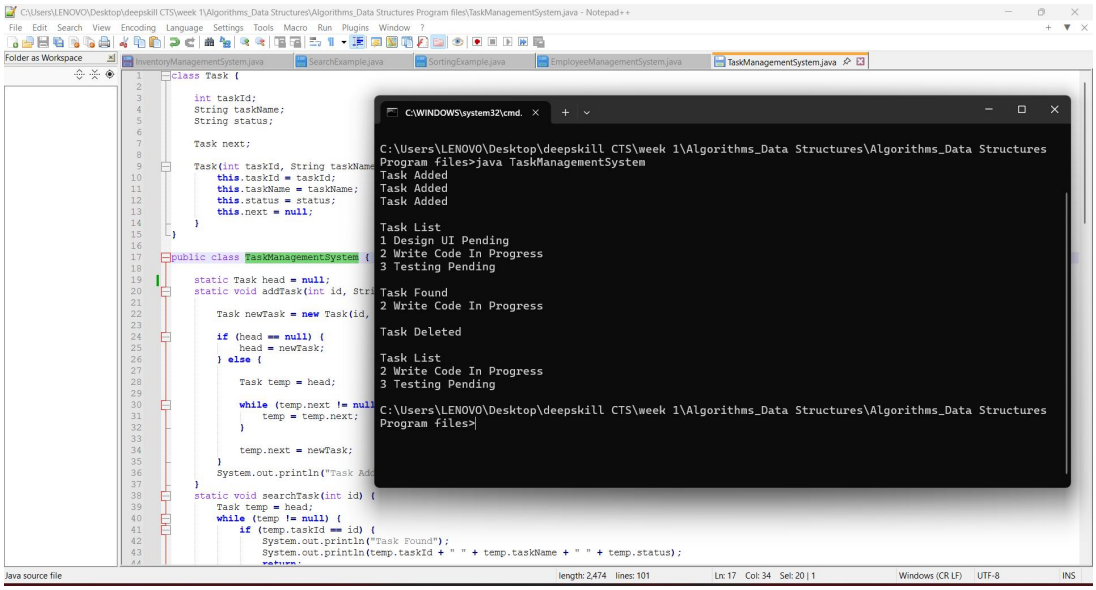
Exercise 5: Task Management System

```
class Task {

    int taskId;
    String taskName;
    String status;

    Task next;
    Task(int taskId, String taskName, String status) {
        this.taskId = taskId;
        this.taskName = taskName;
        this.status = status;
        this.next = null;
    }
}
public class TaskManagementSystem {
    static Task head = null;
    static void addTask(int id, String name, String status) {
        Task newTask = new Task(id, name, status);
        if (head == null) {
            head = newTask;
        } else {
            Task temp = head;
            while (temp.next != null) {
                temp = temp.next;
            }
            temp.next = newTask;
        }
        System.out.println("Task Added");
    }
    static void searchTask(int id) {
        Task temp = head;
        while (temp != null) {
            if (temp.taskId == id) {
                System.out.println("Task Found");
                System.out.println(temp.taskId + " " + temp.taskName + " " + temp.status);
                return;
            }
            temp = temp.next;
        }
        System.out.println("Task Not Found");
    }
    static void displayTasks() {
        Task temp = head;
        System.out.println("\nTask List");
        while (temp != null) {
            System.out.println(temp.taskId + " " + temp.taskName + " " + temp.status);
            temp = temp.next;
        }
    }
    static void deleteTask(int id) {
        if (head == null) {
            System.out.println("List is Empty");
            return;
```

Supersetid :8003253

```
        }
        if (head.taskId == id) {
            head = head.next;
            System.out.println("Task Deleted");
            return;
        }
        Task temp = head;
        while (temp.next != null && temp.next.taskId != id) {
            temp = temp.next;
        }
        if (temp.next == null) {
            System.out.println("Task Not Found");
        } else {

            temp.next = temp.next.next;
            System.out.println("Task Deleted");
        }
    }
    public static void main(String[] args) {
        addTask(1, "Design UI", "Pending");
        addTask(2, "Write Code", "In Progress");
        addTask(3, "Testing", "Pending");
        displayTasks();
        System.out.println();
        searchTask(2);
        System.out.println();
        deleteTask(1);
        displayTasks();
    }
}
```

OUTPUT:



Supersetid :8003253

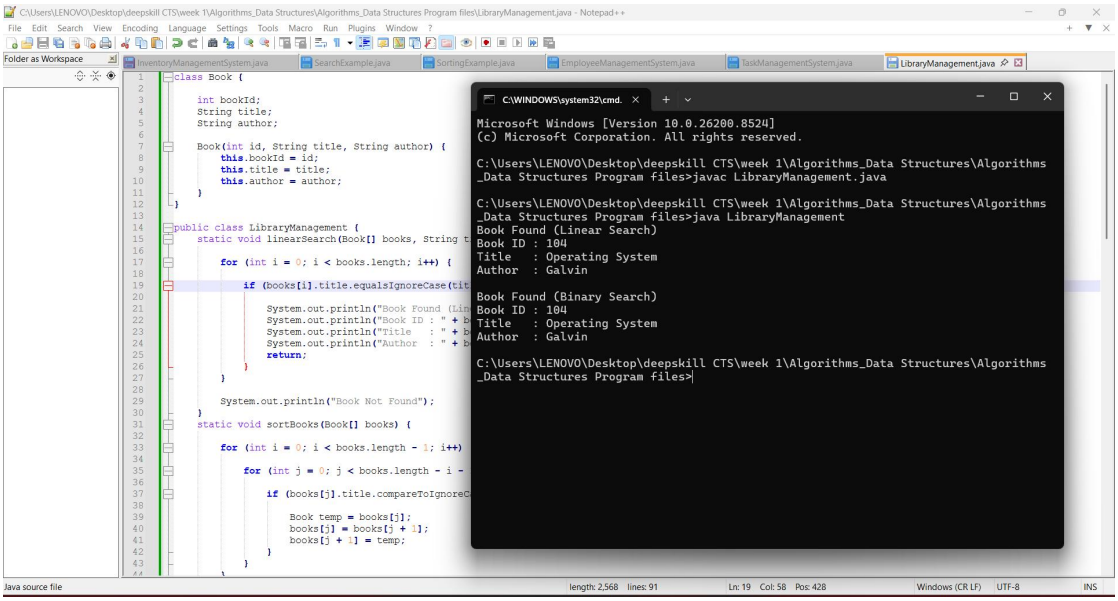
Exercise 6: Library Management System

```
class Book {
    int bookId;
    String title;
    String author;
    Book(int id, String title, String author) {
        this.bookId = id;
        this.title = title;
        this.author = author;
    }
}
public class LibraryManagement {
    static void linearSearch(Book[] books, String title) {

        for (int i = 0; i < books.length; i++) {

            if (books[i].title.equalsIgnoreCase(title)) {

                System.out.println("Book Found (Linear Search)");
                System.out.println("Book ID : " + books[i].bookId);
                System.out.println("Title   : " + books[i].title);
                System.out.println("Author  : " + books[i].author);
                return;
            }
        }

        System.out.println("Book Not Found");
    }
    static void sortBooks(Book[] books) {
        for (int i = 0; i < books.length - 1; i++) {
            for (int j = 0; j < books.length - i - 1; j++) {
                if (books[j].title.compareToIgnoreCase(books[j + 1].title) > 0) {
                    Book temp = books[j];
                    books[j] = books[j + 1];
                    books[j + 1] = temp;
                }
            }
        }
    }
    static void binarySearch(Book[] books, String title) {
        int low = 0;
        int high = books.length - 1;
        while (low <= high) {
            int mid = (low + high) / 2;
            int result = books[mid].title.compareToIgnoreCase(title);

            if (result == 0) {

                System.out.println("\nBook Found (Binary Search)");
                System.out.println("Book ID : " + books[mid].bookId);
                System.out.println("Title   : " + books[mid].title);
                System.out.println("Author  : " + books[mid].author);
                return;
            } else if (result < 0) {
                low = mid + 1;
```

Supersetid :8003253

```
        } else {
            high = mid - 1;
        }
    }
    System.out.println("Book Not Found");
  }
  public static void main(String[] args) {
    Book[] books = {
          new Book(101, "Java", "James"),
          new Book(102, "Python", "Guido"),
          new Book(103, "C Programming", "Dennis"),
          new Book(104, "Operating System", "Galvin"),
          new Book(105, "Database", "Korth")
    };
    linearSearch(books, "Operating System");
    sortBooks(books);
     binarySearch(books, "Operating System");
  }
}
```

OUTPUT:



Supersetid :8003253

Exercise 7: Financial Forecasting

```
public class FinancialForecasting {
   static double futureValue(double amount, double growthRate, int years) {
       if (years == 0) {
          return amount;
       }
       return futureValue(amount * (1 + growthRate), growthRate, years - 1);
   }

   public static void main(String[] args) {

       double presentValue = 10000;
       double growthRate = 0.10;
       int years = 5;

       double result = futureValue(presentValue, growthRate, years);

       System.out.println("Present Value : Rs." + presentValue);
       System.out.println("Growth Rate   : " + (growthRate * 100) + "%");
       System.out.println("Years         : " + years);
      System.out.println("Future Value  : Rs." + result);
   }
```

Supersetid :8003253

Output